AI native engineering assessment

Build a small operations agent that reads company policy, checks a payment status through a tool, gives an evidence-backed answer and proposes a follow-up for human approval. The difficulty is making the AI and the surrounding application behave correctly when evidence is missing, inputs are hostile, calls fail or a worker crashes.

Time and priorities

- **Submission window: 20 hours from receipt of this pack**, once the model-access preflight below is complete. The recruiter confirms the exact start and deadline in writing.

- **Expected focused effort: 8 to 10 hours for the required core. Stop at 12 focused hours**, even if the deadline is later. State time spent and unfinished work prominently. Advanced work is optional; do not work through the night to finish it.

- The core carries 50 points. Depth carries another 50. A smaller correct submission beats a complete-looking unsafe one.

- AI coding tools, documentation, libraries and frameworks are allowed. Explain what you used, what you checked and what you changed. You must explain and modify your own submission in a follow-up call.

This uses fictional tenants and synthetic payments. It is an assessment, not production work. No real payments, blockchain, external email or deployment is required.

Preflight before the clock starts

You need Docker Compose, Python 3.11+ for the provided host scripts, and enough resources for PostgreSQL plus a small API and worker. A machine with 4 cores, 8 GB RAM and 5 GB free disk is sufficient for the supplied fixtures; a local model may need more. Report your machine. A hosted model is allowed without a GPU.

We must arrange usable model access before the start, or you may use a local model you already have. You are not expected to purchase API credits or GPU access. A local deterministic model fixture is included for systems testing, but cannot replace the real-AI demonstration. If model access breaks, notify the recruiter; do not report fixture results as AI results.

What is supplied

`kit/data/` has two tenants, active and superseded policies, an untrusted post containing hostile instructions, payment states, fixed local auth tokens, 16 labelled development cases and 12 labelled public audit cases. The same payment ID can exist with different statuses in the two tenants. Documents and payment records use tenant-scoped identities.

`compose.yaml` starts PostgreSQL and the kit gateway. `kit/mock_gateway.py` supplies a model fixture, read tool, idempotent write tool, persistent audit records and controlled faults. `API_CONTRACT.md` defines your application boundary. `MODEL_PROTOCOL.md` defines the fixture interface and provider configuration. `kit/evaluate.py`, `check_core.py` and `load.py` are host-side probes. They do not implement your application.

Start the infrastructure:

```
cp .env.example .env
# Only local fixture values are supplied; keep actual keys out of Git.
docker compose up -d --build db kit
curl http://localhost:8091/healthz
```

Add services named `api` and `worker` to compose.yaml. API host port is 8090. Keep model environment variables configurable on the worker. Bind the selected data folder read-only at `/assessment-data` in your application services; use `DATA_DIR=/assessment-data`. The host source is `${ASSESSMENT_DATA_DIR:-./kit/data}`. `make seed` must idempotently import that folder. No hidden data formats will be introduced.

Use TypeScript or Python. PostgreSQL is required for durable application state. A single API, a worker and PostgreSQL are enough. You do not need several agents, a vector database, a separate message broker, a paid framework, Kubernetes or a custom frontend design system. Keyword/BM25/SQL retrieval is acceptable if it retrieves and grounds correctly; choose more complexity only when it helps.

Required core worth 50 points

Core area	Points
Grounded AI and honest evaluation	16
Agent control and validated tool use	10
Durable Web2 backend	12
Tenant authorization and approved action	8
Reproducibility and meaningful tests	4

Grounded AI application

Retrieve relevant evidence from the authenticated tenant's **active policy** documents. Do not place the entire cross-tenant corpus in the model context and ask the model to enforce authorization. Superseded and untrusted content must not become current policy. Cite an exact quote, document ID and version, and ensure the answer follows from that evidence.

Use an actual model to produce and validate structured results. For payment requests, the model selects get_payment_status, your code validates and executes that read tool, and the model uses its returned status in the answer. For policy questions, answer from retrieved policy. For unsupported policy or a missing payment, say that human review is needed; invent no facts or permissions. The same backend and validation must run in fixture and real-model modes.

Follow the exact result fields and business rules in API_CONTRACT.md. A model-selected tool or model-generated JSON is untrusted input. Reject unexpected tool names, extra tenant arguments, mismatched payment IDs and malformed results. Do not execute arbitrary URLs, shell commands, SQL or code from the model or retrieved documents.

Durable Web2 backend

Implement create-run, get-run, trace and approve routes from API_CONTRACT.md. A 202 means the request and recoverable work are already durable. Use database-enforced tenant-scoped request idempotency. Concurrent duplicate submissions must yield one run; conflicting reuse must return 409. Preserve results, approvals and action execution through API and worker restarts. Reclaim abandoned work within 30 seconds. Invalid input must not strand healthy work.

Each run has finite limits: six model requests, three read-tool calls, at most one malformed-output repair, call timeout at most 10 seconds, active processing deadline at most 60 seconds. Retries count toward the same limits. Honour Retry-After when it fits the deadline. Persist enough budget/attempt state that restarting cannot create an unlimited loop.

Approval and external effect

A held payment can produce a proposed create_followup action. It must **wait for a same-tenant reviewer**. A reader, model, retrieved document or user instruction cannot approve it. Bind approval to the stored action and hash of the exact immutable arguments. Execute once using a stable external idempotency key, even across concurrent approvals, a worker crash and a tool that commits then drops its response. Paid/missing payments must not produce executable actions.

This is a synthetic follow-up record, not a payment. The tool refuses invalid writes, but your application must enforce its own policy and authorization too.

Evidence and tests

Return a persisted, tenant-scoped trace of retrieval, model attempts, selected tools, validation, approval and action receipts. Do not store secret headers, keys or private model chain-of-thought. Write tests against the production functions you actually use, including at least tenant filtering, duplicate/conflicting requests, schema/tool rejection and approval/idempotency. Demonstrate a worker restart with accepted work outstanding.

Run the real-model evaluation. Report development results honestly; then freeze the final prompt/configuration and run the public audit set once, recording the commit/config hash. If you tune afterwards, retain the original result and label subsequent results as tuned. The public audit set is visible; it is not a secret held-out benchmark. Evaluators also use private variants and changed facts. Merely citing a document ID does not prove an answer is grounded: manually inspect at least three answers and include two errors or useful boundary cases.

Optional depth worth 50 points

Choose a few areas after the core works. There is no requirement to finish all of these.

- **AI quality and evaluation, 12:** improve retrieval, handle paraphrases and mixed-language requests, measure retrieval and citation quality separately from task success, add meaningful regression cases, and explain repeated-run variability.

- **Concurrency and failure recovery, 12:** demonstrate two workers, concurrent request/approval races, crashes during external execution, queue recovery and exact effect counts. Include a reproducible fault test rather than only an architecture description.

- **Resource use and scaling, 10:** bounded queue admission/backpressure, indexes and query plans, per-run token/cost accounting with actual model rates or clearly labelled estimates, measured acknowledgement versus completion latency, and a bounded load test. Fixture tokens are synthetic and not real costs.

- **Review experience, 8:** add a minimal UI for submitting a request, inspecting evidence/status, approving the exact proposal and seeing the receipt. It must use the same auth and API. A simple clear UI earns more than a polished UI over broken backend behaviour.

- **Adaptation and design, 8:** clean provider/tool interfaces, a concrete scaling note and a small demonstrated change without breaking the core. No extra frameworks or agent count earns points on its own.

Commands and checks

Your submission must support:

Command	Expected result
`docker compose up -d --build`	Starts the whole app from a clean checkout; health ready within five minutes, excluding first image download.
`make seed`	Imports the selected fixture folder, safely on repeat.
`make test`	Runs your real application tests inside containers; only Docker and make needed for this command.
`python3 kit/check_core.py`	Public API/auth/approval/idempotent-effect smoke checks, with the app in fixture mode.
`python3 kit/evaluate.py --out results/dev.jsonl`	Development evaluation through the real application API.
`python3 kit/evaluate.py --cases kit/data/audit.jsonl --out results/audit.jsonl`	Declared final public audit run, in actual-model mode.

The public smoke checker is necessary evidence, not a complete grader. It does not establish crash durability, all injection cases, semantic grounding or real-model competence. We inspect code, change facts, run independent cases and perform failure drills.

We may kill/restart api and worker, submit duplicate requests and approvals concurrently, change tenant-scoped documents/payment facts, inject 429s, malformed model output, tool requests with unauthorized arguments, loops and external lost responses. All use the documented interfaces and limits. We will not require an undisclosed infrastructure service or UI for the core.

Submission

Return a private repository with access granted, or a ZIP containing the solution and unchanged kit. Do not submit .env secrets, node_modules, virtual environments, model weights or Docker volumes. Keep normal commit history if using a repository; we do not grade keystrokes or penalize disclosed AI-generated code.

Include:

1. `README.md`: time spent, unfinished work, run instructions, machine, architecture and main guarantees.

2. `EVAL.md` and raw results: exact actual model/configuration, development and frozen public audit results, three inspected examples, limitations, latency and cost arithmetic. Fixture versus actual-model measurements must be clearly separated.

3. `DESIGN.md`, at most 600 words: what breaks at one million requests/day; peak assumption, model concurrency and token cost; queue/DB bottleneck; the metrics you would watch; your next change. One million/day is about 11.6 requests/second average, not one million concurrent users. Use an explicit peak assumption.

4. `AI_USAGE.md`: tools used, one important error you found in generated work, what you changed/rejected and how you verified the final code.

5. Tests, migrations, code, compose configuration and a short recorded or written replay of one failure/recovery drill. No video is required if logs and commands are clear.

How a score of 50 is interpreted

A submission can advance at **50/100 overall**, but needs at least **42/50 core points**, including at least **12/16 in grounded AI**, no critical gate failure and a successful ownership call. We cannot compensate for a data leak, unapproved action, lost acknowledged work, unbounded agent or a fixture-only AI demo with UI or advanced points.

After the take-home, expect a 40-minute walkthrough: explain one run and its transaction boundaries, reproduce a failure, then make and test a small change. AI tools were allowed in the take-home; in the discussion you must first explain your design in your own words. Any live tool use is agreed at the start. Passing is strong evidence for this role; it is not a guarantee of competence in every technical specialty.